Mini-Project: Customers Like You

PCA or SVD for a real retail problem | One week | Pairs or individuals | You are the Data Engineer and the ML Engineer

How to read this brief. Section 1 is the situation you walk into at work. Read it before anything else, because every decision you make should answer the stakeholder's request. Sections 2 to 4 give you the data and the method choice. Sections 5 to 7 tell you exactly what to build, when, and how you will be graded. Project rules are labelled **R1**, **R2** and so on; tasks are labelled **DE** (Data Engineer), **ML** (ML Engineer) and **J** (joint). Keywords are in **red**.

Item	Detail
Duration	One week (7 days), from the day the brief is released
Team	Pairs (one Data Engineer, one ML Engineer) or individuals (both roles)
Method	PCA or SVD, your choice, justified with evidence
Data	UCI Online Retail II, 1,067,371 real invoice lines, Dec 2009 to Dec 2011. **Download it first:** https://archive.ics.uci.edu/dataset/502/online+retail+ii
Grade	60% team product, 40% individual contribution (Section 7)

1. The Scenario

1.1 The Company

DEFINITION

GiftLane is a UK online retailer of gift-ware: candle holders, bunting, lunch bags, paper goods. Most buyers are **small shops and wholesalers** who reorder the same kinds of items several times a year. The company has about 5,900 registered customers in roughly 40 countries and sells several thousand products.

EXAMPLE

A typical order: a gift shop in Lyon buys 24 jumbo shopper bags, 12 heart-shaped T-light holders and 48 packs of paper napkins. Three months later it orders again, mostly similar items. That repeat behaviour is exactly what a recommendation system can learn from.

1.2 Your Team

DEFINITION

You have just joined GiftLane's Data & ML team. In a pair, one of you is the **Data Engineer**: you own the raw data, the cleaning and the matrix the model is built on. The other is the **ML Engineer**: you own the compression method, the recommendations, the evaluation and the customer map. If you work alone, you hold both roles.

EXAMPLE

In a real company these two people work closely, but each is accountable for their own part. The Data Engineer's mistakes (a cancelled order counted as a purchase) flow straight into the ML Engineer's results, so you will need to talk every day.

1.3 The Request from the Stakeholder

On Monday morning this arrives in the team inbox:

From: Sarah Okafor, Head of E-Commerce
To: Data & ML Team
Subject: Two things I need before the Q4 planning meeting

Hi team,

1. Product pages. I want a "Customers who bought this also bought..." box on every product page. The old prototype compares every customer with every product. The overnight job is too slow and the table is too big to keep. Can you shrink the data so the job runs nightly and still gives sensible suggestions? I would like to see that the suggestions are actually better than just showing our best sellers.

2. Marketing. The marketing team wants one simple picture of our main customer groups so they can plan a different campaign for each group.

I am not technical, so please tell me in plain words: which method you chose and why, what we lose by compressing, how big and how fast the result is, and what could go wrong. Customer IDs must not appear in anything that leaves your team.

I need a one-page summary and a short presentation by the end of next week.

Thanks,
Sarah

1.4 What Success Looks Like

DEFINITION

Translate Sarah's email into measurable targets before you start. Your project succeeds if you can show, with numbers: (1) the compressed data is **much smaller** than the full matrix, (2) the nightly job is **fast**, (3) your recommendations **beat a best-seller baseline** on purchases the model never saw, and (4) the customer map shows **groups marketing can name**.

EXAMPLE

Example of a measurable claim for the memo: "With 50 components the product data shrinks from 189 MB to 4 MB, the job runs in 6 seconds, and the correct product appears in the top 10 suggestions for 21% of customers, against 9% for best sellers." (These numbers are illustrations; yours will differ.)

Project rules	
R1	Every decision (a cleaning step, the method, the value of k) is **written down with its reason** in the notebook or the decision log.
R2	Hold out test purchases **before** fitting anything. No model, scaler or decomposition may see them.
R3	Compare against a **baseline**: recommendations from the 10 best-selling products.
R4	**No customer IDs** in the memo, the slides or any figure. Use segment names or anonymised indices.
R5	All work lives in **one shared Git repository**. Each member commits their own work under their own name (Section 7).
R6	Results must be **reproducible**: fixed random seeds, a requirements file, and one command that rebuilds the matrix from the raw file.

2. The Data

Step 0 - Download the data before you do anything else
Each team member downloads the data on Day 1, before the first meeting, and checks that it opens. Do not wait until the day you need it: the file is large, and a failed download should not cost you a day of a one-week project.
1. Open the dataset page: https://archive.ics.uci.edu/dataset/502/online+retail+ii
2. Click **Download**. You get a zip containing **online_retail_II.xlsx** (43.5 MB, two sheets: 2009-2010 and 2010-2011).
3. Put the file in a **data/raw/** folder of your repository. Add data/ to **.gitignore**: raw data is not committed to Git.
4. Check it loads: pd.read_excel("data/raw/online_retail_II.xlsx", sheet_name=None) should return two sheets with about 1.07 million rows in total. Note the row count in your README.
Alternative in Python: pip install ucimlrepo, then from ucimlrepo import fetch_ucirepo; fetch_ucirepo(id=502).
Slow laptop? The smaller original Online Retail dataset (541,909 rows, Dec 2010 to Dec 2011, 22.6 MB) has the same columns: https://archive.ics.uci.edu/dataset/352/online+retail. Tell the instructor if you use it; your time-based split will have one year of history instead of two.

GiftLane's transaction log is the public **Online Retail II** dataset from the UCI Machine Learning Repository: 1,067,371 invoice lines from a real UK online gift-ware retailer, 1 December 2009 to 9 December 2011. Licence CC BY 4.0. The download steps are in the box above. The Excel file has two sheets, one per year; you need both.

Column	Meaning	Type	Watch out for
Invoice	6-digit invoice number	text	Starts with "C" for a cancellation
StockCode	5-digit product code	text	Letters on the end for variants; codes such as POST, M, D, BANK CHARGES are not products
Description	Product name	text	Missing values; the same code can have slightly different names
Quantity	Units on this line	integer	Negative values (returns, adjustments)
InvoiceDate	Date and time of the invoice	datetime	Two years; needed for a time-based hold-out
Price	Unit price in pounds sterling	decimal	Zero or negative prices
Customer ID	5-digit customer number	number	Missing on roughly a fifth of lines; private (R4)
Country	Customer's country	text	Useful for describing segments

Column names in the file can differ slightly from the UCI web page (for example Invoice or InvoiceNo, Price or UnitPrice). Check them when you load the file.

2.1 The Matrix You Will Build

DEFINITION

The Data Engineer turns the log into a **customer x product matrix** A: one row per customer, one column per product. Cell A[i, j] records how much customer i bought of product j. Most customers buy a small fraction of the catalogue, so almost every cell is zero: the matrix is **sparse**.

EXAMPLE

Size check you should reproduce: with about 5,900 customers and 4,000 products, a dense matrix of 8-byte numbers needs 5,900 x 4,000 x 8 = **189 MB**. Stored sparse, only the non-zero cells are kept. After compression to k = 50 components you store customer factors (5,900 x 50) and product factors (4,000 x 50): 9,900 x 50 x 8 = **about 4 MB**.

Choices the Data Engineer must make and justify (R1): what counts as a purchase (raw quantity, a 1/0 "ever bought" flag, or log(1 + quantity)); which products to keep (for example only those bought by at least 10 customers); and what to do with returns, cancellations and non-product codes.

3. PCA or SVD? Know the Difference

Both methods compress A into a few components, and they are closely related: **PCA is SVD applied to centred data**. On a dense table of measurements they give the same directions. On a big sparse purchase matrix the small difference, centring, becomes a big practical difference.

3.1 Singular Value Decomposition (SVD)

DEFINITION

Any matrix can be written as $A = U\,S\,V^T$. The columns of **U** describe the rows (customers), the columns of **V** describe the columns (products), and the **singular values** in S say how important each component is. **Truncated SVD** keeps only the top k and gives the best possible rank-k approximation of A.

EXAMPLE

scikit-learn's TruncatedSVD works **directly on the sparse matrix** and does **not** centre it. You get customer factors (U S) and product factors (V) in one fit: exactly what a recommender needs. This is the classic approach behind latent-factor recommenders and latent semantic analysis for text.

3.2 Principal Component Analysis (PCA)

DEFINITION

PCA first **subtracts each column mean** (and often scales), then finds the directions of maximum variance. Those directions are the right singular vectors of the centred matrix, and each comes with a clear **share of variance explained**.

EXAMPLE

Centring a purchase matrix turns its zeros into small negative numbers, so the matrix is **no longer sparse**. Modern scikit-learn (1.4 and later) can run PCA on sparse input with the arpack or covariance_eigh solvers by centring implicitly, but the result still describes deviations from the average customer. Find out what that means for your recommendations.

3.3 Side by Side

Question	PCA	Truncated SVD
What does it decompose?	The centred (often scaled) data	The data as it is
Centres the columns?	Yes, always	No
Sparse input?	Only with special solvers; centring breaks sparsity	Yes, natively
What you get	Components, scores, explained variance ratio	U, S, V: customer and product factors together
Meaning of component 1	Main direction of variation around the average customer	Often "overall volume", how much a customer buys
Easiest number for a stakeholder	"k components keep 90% of the variance"	"k components rebuild the matrix with X% error"
Typical use	Dense measurements, visualisation, preprocessing	Sparse counts, text, ratings, recommenders

Your decision
Choose one method as your main model and justify it in the memo. Then run the other method on the same matrix with the same k, and fill in the comparison table in task ML5. A good report explains the difference with your own numbers, not only with the definitions above.

4. Tasks by Role

Tasks marked **Core** are required for everyone. Tasks marked Stretch are required for pairs and optional for individuals, so an individual is not doing twice the work of one pair member.

4.1 Data Engineer

Task	What to do	Output	Level
DE1	Load both sheets, check column names and types, report row counts and missing values per column	Data profile table	Core
DE2	Clean: remove cancellations, non-positive quantity or price, non-product codes, lines without a customer ID. Log how many rows each rule removed	Cleaning log	Core
DE3	Split by time: the last 2 months are the test period. For each test customer, hold out purchases of products they never bought before (R2)	Train and test files	Core
DE4	Build the sparse customer x product matrix from training data only, with your chosen purchase value and product filter	Sparse matrix (.npz) and index maps	Core
DE5	Report shape, number of non-zeros, sparsity %, memory dense vs sparse	Matrix report	Core
DE6	Package DE1 to DE5 as one script: python build_matrix.py rebuilds everything from the raw file (R6)	build_matrix.py	Stretch

4.2 ML Engineer

Task	What to do	Output	Level
ML1	Choose PCA or Truncated SVD and write a short justification, before running anything	Decision note	Core
ML2	Fit on the training matrix for k in 10, 20, 50, 100, 200. Plot explained variance (PCA) or reconstruction error (SVD) against k	Curve and chosen k	Core
ML3	Recommend: for each product, find the 10 most similar products by cosine similarity of their factor vectors. For a customer, score products by similarity to what they bought	Recommendation function	Core
ML4	Evaluate on held-out purchases: hit rate at 10 for your model and for the best-seller baseline (R3). Time the nightly job	Results table	Core
ML5	Run the other method with the same k on the same matrix. Fill in the comparison table below	PCA vs SVD table	Core
ML6	Customer map: project customers on 2 components, cluster into 3 to 6 groups (for example k-means), describe each group with its top products and countries	Figure and segment names	Core
ML7	Sensitivity: does the purchase value (count, 1/0, log) change your hit rate?	Short analysis	Stretch

4.3 Joint Tasks

Task	What to do	Output
J1	Agree the success targets from Section 1.4 on Day 1 and write them at the top of the README	README
J2	One-page memo to Sarah in plain language: method and why, what is lost, size and speed, risks, next steps	memo.pdf
J3	5-minute presentation; each member presents the part they owned	Slides

4.4 Comparison Table to Fill In (ML5)

Measure	PCA	Truncated SVD
k used		
Fit time (seconds)		
Peak memory during fit (MB)		
Size of stored factors (MB)		
Variance explained or relative reconstruction error		
Hit rate at 10 on held-out purchases		
Best-seller baseline hit rate at 10		
What component 1 represents, in words		

5. The Week

Day	Data Engineer	ML Engineer	Checkpoint
1	Download the data (Section 2, Step 0); DE1 profile it	ML1 read Section 3, draft the method decision; J1 targets	Repository created, README with targets
2	DE2 cleaning and cleaning log	Prototype ML2 on a small random matrix	
3	DE3 split, DE4 sparse matrix	ML2 on the real training matrix	**Check-in with instructor (10 min)**
4	DE5 report, DE6 script	ML3 recommendations, ML4 evaluation	Matrix frozen: no more cleaning changes
5	Review ML results for data issues	ML5 comparison, ML6 customer map	
6	Both: J2 memo and J3 slides	Both: J2 memo and J3 slides	Peer evaluation submitted
7	Presentation and individual questions	Presentation and individual questions	**Final submission**

Individuals follow the same calendar and do both columns, skipping the Stretch tasks if they wish.

6. Starter Code and Metrics

These pieces get you moving. They are not a complete solution, and you are expected to change them.

```
import numpy as np, pandas as pd, scipy.sparse as sp, time
from sklearn.decomposition import TruncatedSVD, PCA
from sklearn.preprocessing import normalize

# DE1: load both sheets (check the column names first!)
sheets = pd.read_excel("online_retail_II.xlsx", sheet_name=None)
df = pd.concat(sheets.values(), ignore_index=True)
print(df.columns.tolist(), len(df))

# DE4: sparse customer x product matrix from TRAINING rows only
cust = {c: i for i, c in enumerate(train["Customer ID"].unique())}
prod = {p: j for j, p in enumerate(train["StockCode"].unique())}
rows = train["Customer ID"].map(cust); cols = train["StockCode"].map(prod)
vals = np.log1p(train["Quantity"])                # one possible purchase value
A = sp.csr_matrix((vals, (rows, cols)), shape=(len(cust), len(prod)))
print(A.shape, A.nnz, f"sparsity {1 - A.nnz / (A.shape[0] * A.shape[1]):.2%}")
```

```
# ML2: Truncated SVD on the sparse matrix (no centring)
t = time.time()
svd = TruncatedSVD(n_components=50, random_state=42).fit(A)
print("fit seconds", time.time() - t)
P_vec = normalize(svd.components_.T)              # one row per product, length 1

# ML5: PCA on the same matrix (implicit centring, needs scikit-learn 1.4+)
pca = PCA(n_components=50, svd_solver="arpack", random_state=42).fit(A)
print(pca.explained_variance_ratio_.sum())

# ML3: 10 products most similar to product j
def similar(j, n=10):
    s = P_vec @ P_vec[j]
    s[j] = -np.inf
    return np.argsort(-s)[:n]
```

Metric	Definition	Why the stakeholder cares
Hit rate at 10	Share of test customers for whom at least one held-out product appears in their top 10 recommendations	Do the suggestions match what people really bought next?
Baseline hit rate	The same, recommending the 10 best sellers they have not bought	Is the model worth building at all?
Relative reconstruction error	||A - A_k|| / ||A||, using the Frobenius norm	What is lost by compressing
Explained variance ratio	Sum of the kept eigenvalues over the total (PCA)	Same question, in PCA's language
Storage and time	Size of stored factors in MB; seconds to fit and to score all products	Will the nightly job fit?

7. Grading and How Contribution Is Measured

Each person's grade has two parts: **60% team product**, shared by the team, and **40% individual contribution**, which can differ between partners. Contribution is not measured by one thing. It is measured by **four independent sources of evidence**, so no single source (a padded commit log, a generous partner, a nervous presentation) decides a grade on its own.

How contribution is measured: four sources of evidence	
C1	**Repository history.** Every member commits their own work from their own account, with clear messages (R5). We look at what was built and by whom, not at the number of commits or lines: ten tiny commits do not beat one solid one.
C2	**Contribution log.** A shared table, updated daily, listing each task, its owner, the date and the commit or file that shows it. Both partners sign it at the end.
C3	**Confidential peer evaluation.** On Day 6 each member rates themselves and their partner on five criteria (form below). The instructor reads it in private and uses it to adjust the team score for each person.
C4	**Individual questions.** After the presentation each member answers two or three questions alone, including one about their partner's part. Owning a task means being able to explain it.

7.1 The Formula

Final grade = 0.60 x Team score x Peer factor + 0.40 x Individual score.

- **Team score** (out of 100) comes from the rubric in 7.2 and is the same for both partners.
- **Peer factor** comes from C3: your average rating from your partner divided by the team's average rating, kept between **0.80 and 1.05**. Equal contributors both get 1.00. If the ratings disagree badly, the instructor checks C1 and C2 before applying anything.
- **Individual score** (out of 100): 50 points from C4 (questions), 30 from the owned tasks in C1 and C2 (completeness and quality), 20 from the individual reflection (half a page: what you did, what you learned, what you would change).
- **Individuals** have a peer factor of 1.00. Their individual score uses C1, C4 and the reflection in the same way.

Worked example
Team score 85. Partner A is rated 4.6 by B; B is rated 3.8 by A; team average (4.6 + 3.8) / 2 = 4.2. A: factor 4.6 / 4.2 = 1.10, capped at 1.05. B: factor 3.8 / 4.2 = 0.90. With individual scores A 88 and B 80: A = 0.60 x 85 x 1.05 + 0.40 x 88 = 53.55 + 35.2 = 88.75. B = 0.60 x 85 x 0.90 + 0.40 x 80 = 45.9 + 32.0 = 77.9.

7.2 Team Rubric (100 points)

Criterion	Points	Full marks look like
Data engineering (DE1 to DE6)	25	Clean, logged, leak-free split; sparse matrix with a clear report; pipeline reruns in one command
Method choice and PCA vs SVD comparison (ML1, ML5)	20	Choice justified before fitting; differences shown with own numbers, not only definitions
Model and evaluation (ML2 to ML4)	20	k chosen with evidence; hit rate beats or honestly loses to the baseline, explained
Customer map (ML6)	10	Readable figure; segments named from their products and countries; no customer IDs
Memo to the stakeholder (J2)	15	One page, plain language, answers all of Sarah's questions with numbers and risks
Reproducibility and code quality (R6)	10	Seeds, requirements file, clear notebook, README explains how to run

7.3 Peer Evaluation Form (C3)

Rate yourself and your partner from 1 (rarely) to 5 (always). Add one sentence of evidence for any rating of 2 or below.

Criterion	Me	Partner
Completed my agreed tasks on time		
Quality of work (correct, tested, documented)		
Communicated and replied within a day		
Helped the other role understand my part		
Contributed to the memo and presentation		
Average		

7.4 Contribution Log Template (C2)

Date	Task	Owner	Evidence (commit, file or figure)	Status
Day 1	DE1 data profile			
Day 1	J1 success targets			
...	...			

8. Submission Checklist

Submit one link to the repository and upload the memo and slides to Canvas by the end of Day 7.

- ☐ Data downloaded on Day 1 and its row count noted in the README (Step 0)
- ☐ README with targets, how to run, and team roles
- ☐ build_matrix.py (or notebook section) that rebuilds the matrix from the raw file
- ☐ Analysis notebook, run top to bottom without errors
- ☐ Cleaning log and matrix report (DE2, DE5)
- ☐ PCA vs SVD comparison table filled in (ML5)
- ☐ Customer map figure with named segments and no customer IDs (ML6, R4)
- ☐ memo.pdf, one page, for Sarah (J2)
- ☐ Slides (J3)
- ☐ Contribution log signed by both partners (C2)
- ☐ Peer evaluation form, submitted privately (C3)
- ☐ Individual reflection, half a page, one per person

Data source: Chen, D. (2012). Online Retail II [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CG6D. Licence CC BY 4.0.
GiftLane and Sarah Okafor are fictional; the transactions are real.